Q1 [8 Points]

Use the inverse transform method to derive a simulation algorithm for generating samples from the distribution.

- Use `sympy` to help you derive the simulation algorithm as a function of u (the CDF sample) and θ.
- Generate 5,000 samples from this distribution with $\theta = 4$ and report the average of these samples.

In [19]:
```
x, u, theta= sympy.symbols("x, u, theta", positive=True)
```

In [20]:
```
fx = theta*x**(-1-theta)
cdf = sympy.integrate(fx, (x, 1, x))
sympy.solve(cdf-u,x)
```

Out[20]:
$$\left[\left(-\frac{1}{u-1}\right)^{\frac{1}{\theta}}\right]$$

In [21]:
```

```

In [22]:
```
u = np.random.sample(5000)
samples = (-1/(u-1))**(1/4)
samples
```

Out[22]:
```
array([1.44608313, 1.00525634, 1.28536192, ..., 1.00742152, 1.52064687,
       1.09806407])
```

In [23]:
```
np.mean(samples)
```

Out[23]:
1.32281809228658

Q2 [8 Points]

Let's use MLE to estimate θ using the samples (given by `samples_Q2`) drawn from the distribution.

- Use `sympy` to derive the MLE formula for estimation as a function of n (defined as `n`) and $\sum log(x_i)$ (defined as `Slogx`). Make sure to check the second order condition.
- Based on the formula, report $\hat{\theta}$.

In [24]:
```
samples_Q2
```

Out[24]:
```
array([1.44608313, 1.00525634, 1.28536192, ..., 1.00742152, 1.52064687,
       1.09806407])
```

In [25]:
```
Slogx, n, theta = sympy.symbols("logx_sum, n, theta")
JLL = n*sympy.log(theta) - (1+theta)*Slogx
sympy.solve(sympy.diff(JLL,theta),theta)
```

Out[25]:
$$\left[\frac{n}{logx_{sum}}\right]$$

In [26]:
```
sympy.diff(JLL,theta, theta)
```

Out[26]:
$$-\frac{n}{\theta^2}$$

In [27]:
```
len(samples_Q2)/np.sum(np.log(samples_Q2))
```

Out[27]:
$$4.115385207205 28$$

Q3 [6 Points]

Verify whether the population mean μ of the distribution that generated the samples is **different from** 1.3.

Clearly show the calculation, without using pre-built t-test functions, of:

- The t-score based on the data.
- The corresponding p-value.

Finally, state your conclusion using the significance level is $\alpha = 0.05$.

In [28]:
```
n = len(samples)
x_bar = np.mean(samples)


se_x_bar = (np.var(samples,ddof=1)/n)**0.5
t= (x_bar - 1.3 )/se_x_bar
t
```

Out[28]: 3.32904292828644

In [29]:
```
2*stats.t.sf(abs(t),df = n-1)
2*stats.t.sf(t,df = n-1)
2*(1-stats.t.cdf(abs(t),df = n-1))
2*(1-stats.t.cdf(t,df = n-1))
```

Out[29]: 0.000877757566576376

In [30]:
```
## Since p-value is less than 0.05, we will the null hypothesis
##  and conclude we have sufficient evidence to support that mu is different from
1.3
```

Q4 [8 Points]

It can be derived that the actual mean of the population distribution is $\frac{4}{3}$.

- Compute β, the probability of making a Type II error, for the hypothesis test performed in Q3.

In [31]:
```
alpha = 0.05
cv1 = stats.t.ppf(alpha/2,df = n-1)
cv2 = stats.t.ppf(1-alpha/2,df = n-1)
p2 = stats.t.cdf(cv2, loc = (4/3-1.3)/se_x_bar,df = n-1)
p1 = stats.t.cdf(cv1, loc = (4/3-1.3)/se_x_bar, df = n-1)
p2-p1
```

Out[31]: 0.00185775125376686

In [32]:

In [33]:

In [34]: